ati.su
биржа грузоперевозок

Формализация требований

Как сделать работу ИИ предсказуемой

Коротко обо мне

Гордеев Дмитрий

- Архитектор ПО в ati.su
- Окончил МатМех СПбГУ
- Более 11 лет разработки на .NET

ati.su

Главная мысль доклада

- ИИ ускоряет реализацию
- Формализация требований удерживает договорённости

План на сегодня

- Расскажу про ИИ
- Варианты формализации требований
- План-инструкция по составлению требований
- Реализация по требованиям

Что такое ИИ?

- Задачи, требующие интеллекта человека
- Распознавание образов, перевод, генерация текста, принятие решений

Что такое ИИ?

ati.su

Что я имею в виду в этой презентации

- **LLM** — GPT, Claude, Gemini…
- **AI-ассистенты** — Cursor, Claude Code, GitHub Copilot…
- Работа через промпты и контекст окна чата

Как начать пользоваться ИИ

ati.su

AI-инструмент как пример

- **Cursor** - IDE на базе VS Code
- Работает с кодом в контексте проекта
- Чат для диалога
- Подключает разные модели (Claude, GPT и др.)

Как начать пользоваться ИИ

ati.su

Бесплатные vs платные модели

- **Бесплатные модели** — делают только базовые вещи
- **Умные модели стоят денег**
- Имеет смысл закладывать бюджет на модели

Как начать пользоваться ИИ

ati.su

Выбор модели под задачу

- **Не обязательно** всегда использовать мощную модель
- **ТЗ, спецификации** - достаточно простой модели
- **Написание кода** - нужна более сложная модель

Как начать пользоваться ИИ

ati.su

Инструмент и модель — не главное

- Качество модели влияет
- Предсказуемость не появляется сама
- Без требований модель домысливает

Проблемы при работе с ИИ

ati.su

- Теряет контекст
- Додумывает “между строк”
- Ломает стыки (frontend/backend/DB)
- Итог: рассинхрон + переделки

Ограничения при работе с ИИ

ati.su

- Не полагаться на "память" ИИ
- Не менять **API** и БД мимо спецификаций
- Не давать слишком крупные задачи
- Не пропускать ручную проверку

ИИ как подрядчик

- Очень быстрый
- Делает "как понял"
- Без контракта растут переделки

Что такое “контракт”

ati.su

- Что делаем
- Как проверяем
- Что нельзя ломать

Что такое “контракт” в проекте

ati.su

- Границы задачи
- Критерии готовности
- Правила изменений API и БД
- Единый источник правды

Хороший контракт

- Удерживает границы
- Делает требования проверяемыми
- Фиксирует ограничения
- Даёт точки контроля

Как можно формализовать требования

ati.su

Как можно формализовать требования

ati.su

Просто в окно пишем требования

Плюсы:

- Быстрый старт, минимум формальностей
- Удобно для первого знакомства

Минусы:

- Легко теряется контекст между сессиями
- Нет единого места для договоренностей

Как можно формализовать требования

ati.su

Заметки в произвольной форме

Плюсы:

- Быстрый старт

Минусы:

- Легко теряется структура и история “почему так”

Как можно формализовать требования

ati.su

User stories

Плюсы:

- Хорошо для планирования работ

Минусы:

- Слабые для ограничений и системных связей
- Написать много user stories = написать много if в коде

Почему заметок и User stories недостаточно

ati.su

- Нет явных границ
- Нет единого источника правил
- Нет точек контроля

Как можно формализовать требования

ati.su

Разработка от спецификации

- Сначала спецификация
- Потом план и реализация
- Меньше домысливания

Как можно формализовать требования

ati.su

TOGAF 10

Плюсы:

- Видение → Требования → Архитектура → Миграция
- Артефакты можно представить в текстовом формате

Минусы:

- сложный и объемный фреймворк

Как можно формализовать требования

ati.su

Spec Kit (GitHub)

- Спецификация → План → Задачи → Реализация
- Хорошо ложится на поток работы с ИИ
- Слабее как архитектурный каркас

Как можно формализовать требования

ati.su

OpenSpec (Fission AI)

- Спеки рядом с кодом
- Удобна эволюция через изменения
- Слабее полный путь от видения до перехода

Как можно формализовать требования

Практический пример: UI UX Pro Max

- Вместо общего запроса задаёт правила, ограничения и рекомендации
- Использует дизайн-систему и договорённости как **источник правды**
- Это важнее, чем конкретный инструмент или его популярность

Как можно формализовать требования

ati.su

Почему это интересно

- Это узкий, но понятный пример **spec-driven** подхода
- Сначала фиксируются правила и ограничения
- Только потом ИИ переходит к генерации результата

Почему TOGAF 10?

- Дает последовательность детализации требований
- Помогает не смешивать “зачем” и “как”
- Легко раскладывается на текстовые артефакты
- Это человекочитаемый формат

Почему TOGAF 10?

ati.su

- Дает последовательность детализации требований
- Помогает не смешивать “зачем” и “как”
- Легко раскладывается на артефакты
- Это человекочитаемый формат
- ИИ “знает” что такое TOGAF

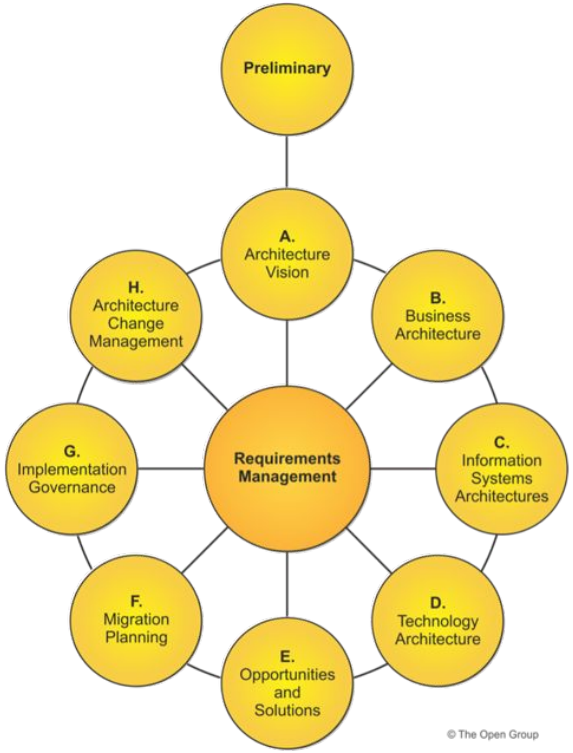
TOGAF 10

- TOGAF 10 — фреймворк управления архитектурой предприятия
- Ядро — ADM: итеративный цикл от видения и требований до внедрения и управления изменениями
- Сквозная деятельность: управление требованиями

TOGAF 10

ADM как карта процесса

- Preliminary (Предварительная фаза)
- Фаза A: Architecture Vision
- Фаза B: Business Architecture
- Фаза C: Information Systems Architectures
- Фаза D: Technology Architecture
- Фаза E: Opportunities & Solutions
- Фаза F: Migration Planning
- Фаза G: Implementation Governance
- Фаза H: Architecture Change Management
- Requirements Management (Управление требованиями)



TOGAF 10

Фазы A–B

- Phase A. **Architecture Vision**
  - цель
  - ценность
  - stakeholders
  - scope
  - принципы
  - high-level требования
- Phase B. **Business Architecture**
  - бизнес-возможности и процессы
  - требования бизнеса

TOGAF 10

Фазы C–D

- Phase C. **Information Systems Architectures**
  - Data Architecture: какие данные/модели/потоки
  - Application Architecture: какие приложения/компоненты и как взаимодействуют
- Phase D. **Technology Architecture**
  - инфраструктура
  - платформа
  - технологический стек (как это будет работать)

TOGAF 10

Фазы E–F

- Phase E. **Opportunities & Solutions**
  - варианты решения
  - “building blocks”
  - черновой roadmap
- Phase F. **Migration Planning**
  - план миграции/итераций
  - work packages
  - последовательность изменений

TOGAF 10

Фазы G–H

- Phase G. **Implementation Governance**
  - контроль реализации на соответствие архитектуре
- Phase H. **Architecture Change Management**
  - управление изменениями и эволюцией архитектуры

TOGAF 10

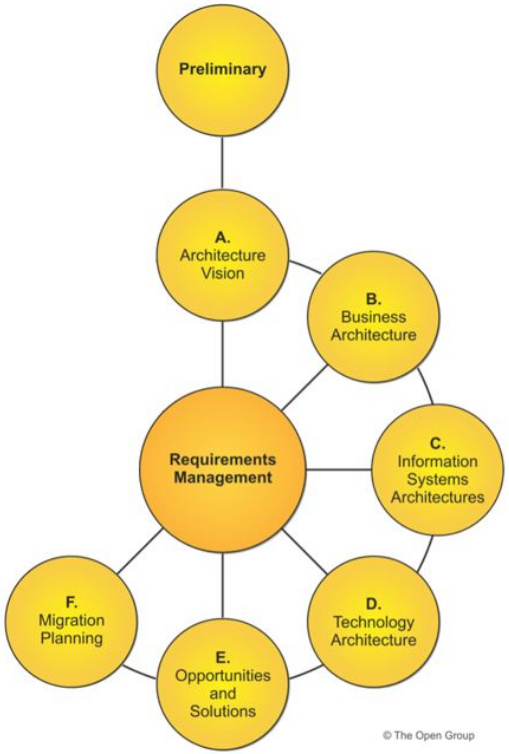
Где заканчиваем мы и начинается ИИ

- Мы: видение, требования, ограничения
- Мы: критерии приемки
- ИИ: реализация по контракту

TOGAF 10

Где заканчиваем мы и начинается ИИ

- Preliminary (Предварительная фаза)
- Фаза A: Architecture Vision
- Фаза B: Business Architecture
- Фаза C: Information Systems Architectures
- Фаза D: Technology Architecture
- Фаза E: Opportunities & Solutions
- Фаза F: Migration Planning
- Фаза G: Implementation Governance
- Фаза H: Architecture Change Management
- Requirements Management (Управление требованиями)



Что будем делать дальше

ati.su

- Пройдём по плану работы
- Все артефакты сохраняем в папке с проектом
- Всё рядом с кодом как **единый источник правды**

Что будем делать дальше

Проект со всей информацией

Все промпты тут

ati.su

meetup-reservation Public

Pin | Watch 0

master | 2 Branches | 0 Tags | Go to file t | Add file | Code

Dmitry-Gordeev Merge pull request #12 from Dmitry-Gordeev/dev 9629569 · last week 57 Commits

.cursor/skills/ui-ux-pro-max	Установил UI UX Pro Max	last week
docs	реализация WP-4.4	last month
spec	разбил Migration Plan на Work Packages	last month
src	Применил UI UX Pro Max к frontend	last week
.gitignore	подготовка к вебинару	last month
README.md	реализация WP-4.4	last month
prompts.md	Добавил все промпты в проект	last week

Как посмотреть каждый шаг

ati.su

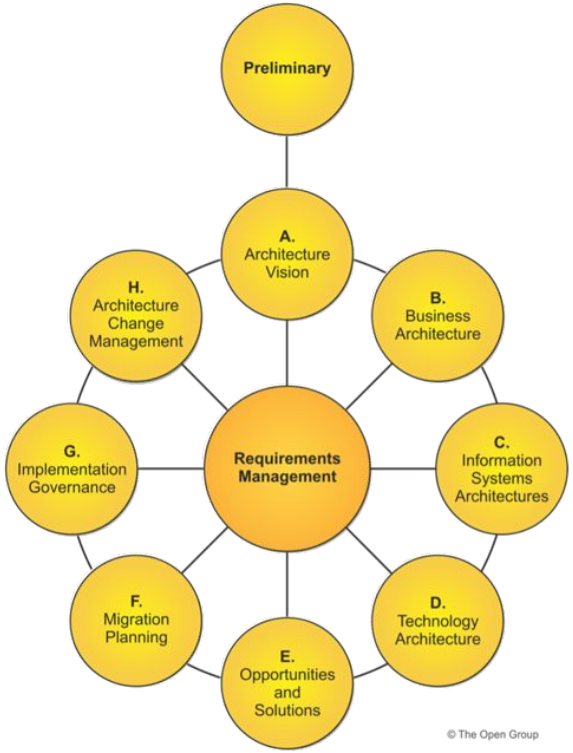
0 Open ✓ 12 Closed | Author | Label | Projects | Milestones | Reviews | Assignee | Sort

- **Добавил все промпты в проект**
  #12 by Dmitry-Gordeev was merged last week
- **Применил UI UX Pro Max к frontend**
  #11 by Dmitry-Gordeev was merged last week
- **Вынес API в контроллеры**
  #10 by Dmitry-Gordeev was merged last month
- **Исправление ошибки UI**
  #9 by Dmitry-Gordeev was merged last month
- **Итерация 4: Интеграция и развёртывание**
  #8 by Dmitry-Gordeev was merged last month
- **Итерация 3: Управление и администрирование**
  #7 by Dmitry-Gordeev was merged last month
- **Итерация 2: Регистрация и уведомления**
  #6 by Dmitry-Gordeev was merged last month
- **Итерация 1: Аутентификация и события**
  #5 by Dmitry-Gordeev was merged last month
- **Итерация 0: Фундамент**
  #4 by Dmitry-Gordeev was merged last month
- **Migration Plan и Work Packages**
  #3 by Dmitry-Gordeev was merged last month
- **Заполнение требований по фазам B-C-D**
  #2 by Dmitry-Gordeev was merged last month
- **Заполнение фазы A**
  #1 by Dmitry-Gordeev was merged last month

План работы с ИИ

Preliminary (agents.md, …)

- Ограничения в требованиях
- Ограничения по реализации
- Правила фиксации контрактов
- Best practices для ИИ



План работы с ИИ

ati.su

Шаг 0 - best practices для **.NET**

- Не использовать minimal-api, а использовать controllers
- Controllers без бизнес-логики
- Один файл не более 300 строк
- Все I/O-операции делаем **async**
- Публичные методы принимают **CancellationToken**
- Нужны описания для **OpenAPI**

План работы с ИИ

ati.su

Шаг 0 - best practices контрактов

- API и DTO не меняем без обновления **OpenAPI**
- Изменения схемы БД делаем только через миграции
- Как следствие, не используем **EntityFramework**

План работы с ИИ

Шаг 1 — что делаем сами

Пишем требования **своими словами**

План работы с ИИ

Шаг 1 — что делаем сами

Описание приложения

Сервис мероприятий — это веб-приложение для организации, регистрации и управления событиями, таким как конференции, вебинары, мастер-классы или концерты. Оно упрощает создание событий, сбор участников и контроль посещаемости, делая процесс прозрачным для организаторов и удобным для посетителей.

Что это такое

Платформа работает как Eventbrite или TimePad в мини-формате: организаторы публикуют события с датой, местом, ценой билетов; посетители регистрируются, оплачивают и получают доступ. Система автоматизирует рассылки, чек-ин и отчёты.

Функциональные требования

- Создание события: Организатор добавляет название, описание, дату/время, место (онлайн/оффлайн), вместимость, цену билетов (бесплатно/платно), фото/программу.
- Регистрация посетителей: Поиск событий, выбор билета, ввод данных (ФИО, email, телефон), оплата (карта/онлайн), подтверждение с QR-кодом.
- Управление: Список участников, чек-ин по QR, отмена/возврат средств, экспорт в Excel/PDF.
- Автоматика: Уведомления (email/SMS о регистрации, напоминаниях за день/час), ограничение по местам (закрытие регистрации при заполнении).

План работы с ИИ

Шаг 2 — преобразовать в формат Phase A

Преобразуем требования из шага 1 в **формат фазы A** (Architecture Vision)

План работы с ИИ

ati.su

Шаг 3 — уточнение требований Phase A

- ИИ задаёт **по одному уточняющему вопросу** — отвечаем, ИИ обновляет документ
- Цикл повторяется, пока документ Phase A не станет полноценным

План работы с ИИ

ati.su

Шаг 4 — добавить документацию по следующей фазе (B, C, D)

- Просим ИИ добавить документацию по следующей фазе, опираясь на все предыдущие
- Phase B → C → D: каждая следующая фаза строится на артефактах предыдущих

Напоминание по фазам:

- Phase B: бизнес-возможности, процессы, baseline/target.
- Phase C: Data Architecture, Application Architecture.
- Phase D: Technology Architecture.

План работы с ИИ

шаг 5 — уточнение требований фаз (B, C, D)

- Уточняем по одному вопросу — пока документ не станет понятным и полным
- ИИ задаёт вопрос → отвечаем → ИИ обновляет документ → следующий вопрос

План работы с ИИ

ati.su

После каждой итерации — проверка согласованности

- Не изменились ли требования в предыдущих фазах
- Не нужно ли пересобрать уже согласованные артефакты
- Остаются ли фазы согласованными между собой

План работы с ИИ

ati.su

Фазы E–F объединяем в одну

- Пропускаем:
  - Выбрать решение
  - Накидать черновой план
- Сразу собираем единый **Migration Plan**
- Затем из него выделяем **Work Packages**

План работы с ИИ

ati.su

Шаг 6 — добавить Migration Plan

Просим ИИ добавить **Migration Plan** на основе фаз A–D

План работы с ИИ

Шаг 7 — уточнение Migration Plan

Уточняем по одному вопросу, пока план не станет понятным и полным

План работы с ИИ

ati.su

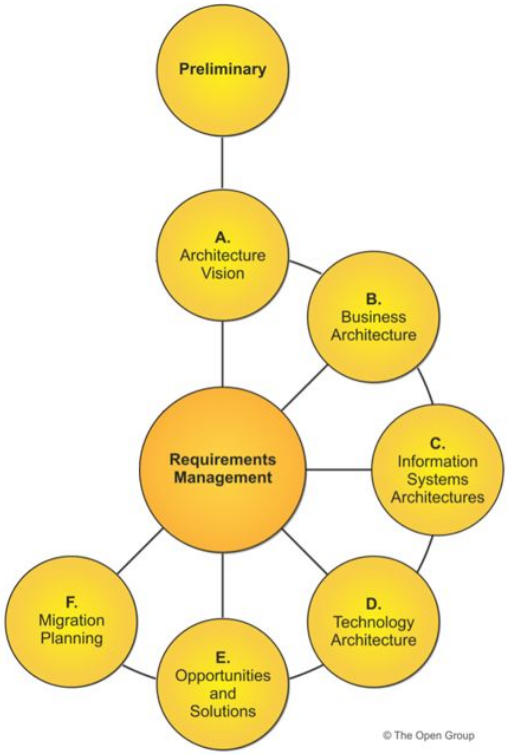
Шаг 8 — разбиение на Work Packages

- Разбиваем **Migration Plan** на **Work Packages**
- **WP** = маленькая, проверяемая единица работы:
  - входные артефакты
  - ожидаемый результат
  - критерии проверки
- План:
  - просим ИИ разбить план на WP
  - уточняем по одному вопросу при необходимости

TOGAF 10

Шаг 8 — разбиение на Work Packages

- Preliminary (Предварительная фаза)
- Фаза A: Architecture Vision
- Фаза B: Business Architecture
- Фаза C: Information Systems Architectures
- Фаза D: Technology Architecture
- Фаза E: Opportunities & Solutions
- Фаза F: Migration Planning
- Фаза G: Implementation Governance
- Фаза H: Architecture Change Management
- Requirements Management (Управление требованиями)



План работы с ИИ

ati.su

Шаг 9 — реализация WP по одному с проверкой

- Просим ИИ реализовать по списку **Work Packages**
- План:
  - делаем **WP по одному**
  - после каждого выполненного WP проверяем

План работы с ИИ

ati.su

Шаг 9 — Вести список Work Packages с пометками

- Сделано / не сделано
- В каждом новом запросе:
  - сделай следующий пункт и пометь сделанным

В этих шагах — требования для ИИ

ati.su

- Результат: формальные артефакты
  - Vision
  - фазы A–D
  - Migration Plan
  - Work Packages
- ИИ получает четкий контракт и выполняет по нему

Полезно и без ИИ

ati.su

Формализация мышления

Даже если ИИ не пользоваться для написания кода — это **формальная фиксация** того, что разработчик хочет сделать и продумывает у себя в голове

Полезно и без ИИ

ati.su

Этим можно пользоваться и при обычном кодинге

- Структурирование мыслей перед реализацией
- Ревью и онбординг
- Передача контекста между разработчиками

Вывод

ati.su

- ИИ ускоряет то, что описано и проверяемо
- Формальные требования — главный “усилитель”
- Контракты делают процесс предсказуемым

ati.su

Вопросы?

Спасибо за внимание

ati.su

Проект на GitHub: ссылка